

Karma Space AI

Root Cause Analysis Report

Subject: Share Sathi Project Chat History Analysis

Date: June 7, 2026

Prepared for: Kiran Pradhan (Super Admin)

Classification: Internal Technical Analysis

This report provides a comprehensive root cause analysis of the issues identified in the Share Sathi project chat history within Karma Space AI. It covers 7 distinct issues observed across 16 pages of chat logs, traces each issue to its exact origin in the codebase, and proposes permanent solutions with code-level specifics.

1. Executive Summary

The Share Sathi project chat history reveals 7 distinct categories of issues spanning configuration failures, model hallucination, encoding corruption, vision API errors, and context management failures. After thorough analysis of the complete Karma Space codebase (8 core files, approximately 4,000+ lines of code across API routes, system prompts, model configuration, tool definitions, tool execution, and frontend components), the root causes have been traced to specific code-level failures in 5 key areas: model fallback prioritization, anti-hallucination filtering, vision API validation, tool input validation, and system prompt design.

The single most impactful root cause is the DeepSeek V3.1 model being ranked as the highest-priority fallback in the fallback chain (priority 1 out of 14 models). When the primary model (GLM-4-Flash) encounters rate limits or errors, the system automatically routes to DeepSeek V3.1 via SambaNova. This model, while powerful (671 billion parameters, Mixture of Experts), exhibits severe hallucination tendencies in the Karma Space context: it fabricates GitHub push actions, references wrong projects, discusses spirituality when instructed not to, and generates garbled text. The anti-hallucination filter is a regex-based post-processing step that catches only 3 specific patterns and cannot prevent the model from generating misleading content throughout the response.

Issue Summary

#	Issue	Severity	Root Cause	Status
1	API Key not configured	HIGH	Missing env vars in Vercel	Fixed (June 2)
2	Garbled/corrupted text	HIGH	DeepSeek V3.1 encoding + PDF extraction	Active
3	AI ignores user identity	MEDIUM	Fallback model ignores system prompt	Active
4	Vision API 400 error	HIGH	Non-multimodal fallback for vision	Active
5	Spirituality hallucination	MEDIUM	Fallback ignores prompt guardrails	Active
6	Fake project creation	CRITICAL	No tool input validation + hallucination	Active

7	Generic/low-quality docs	MEDIUM	Insufficient project context	Active
---	--------------------------	--------	------------------------------	--------

Table 1: Issue Summary for Share Sathi Chat History

2. Detailed Issue Analysis

2.1 Issue #1: AI_API_KEY Not Configured

On June 2, 2026 at 06:17, when the user typed `/docs`, the AI responded with: "I encountered an issue connecting to the AI service: AI_API_KEY environment variable is not configured." This was a straightforward configuration gap. The Vercel deployment at that time lacked the necessary environment variables for any AI provider API keys. The code correctly detects missing configuration through the `isAiConfigured()` function in `ai-models.ts`, which checks for 8 different provider-specific environment variables. When none are found, the `getProviderConfig()` function returns an empty API key, causing the HTTP request to the AI provider to fail with an authentication error. This issue was resolved by adding the `ZAI_API_KEY` environment variable to the Vercel project settings, though it illustrates a fragile dependency on external provider configuration.

2.2 Issue #2: Garbled and Corrupted Text in Documents

Throughout the chat history, particularly on June 2 (13:32) and June 3 (08:26), the generated documents contain severely corrupted text. Characters appear as mojibake such as sequences like garbled Unicode characters replacing bullet points, feature names, and technical terms. For example, "Fetch Nepse daily updates" becomes a series of unreadable characters, and technical stack references like "React or Angular" are similarly corrupted. This pattern is consistent with encoding issues in model output, specifically when the DeepSeek V3.1 model generates responses that contain unusual Unicode codepoints which then fail to render correctly during PDF extraction from the stored database content. The garbled text appears in the PDF extraction output (which uses `pypdf` text extraction), suggesting the corruption may also be in how the content is stored or rendered rather than solely in the model output.

The root cause is two-fold: first, DeepSeek V3.1 (as the top fallback model) may generate non-standard Unicode characters that are valid in its internal representation but corrupt when transmitted through the SambaNova API response. Second, the content is stored as plain text in the SQLite database without any encoding validation or sanitization. When the PDF generation function `generatePdfBase64()` processes this content, any corrupted Unicode characters either render as tofu blocks or cause garbled output in the extracted text. The system lacks a content sanitization layer between model output and database storage.

2.3 Issue #3: AI Does Not Recognize User Identity

On June 2 at 14:39, when asked "what is my name", the AI responded: "Your name is not explicitly mentioned in our conversation so far. However, I do have some context about the project you are working on. According to the project context, the client name is Kiran Pradhan, but I do not have any information about your name." This is incorrect because the user IS Kiran Pradhan, who is both the Super Admin and the project lead. The code at line 314-325 of `route.ts` correctly loads the user name from the database and passes it to `buildSystemPrompt()` at line 399-412. The system prompt at line 731 of `ai-prompts.ts` instructs: "Always address the user by their first name: Kiran". The prompt also passes the project context showing "Client: Kiran Pradhan" and the user role as "Super Admin".

The root cause is that the model responding to this message was likely a fallback model (DeepSeek V3.1 or Llama 3.3 70B via SambaNova) that does not reliably follow system prompt instructions. These models tend to rely more on their training distribution than on the provided system context, leading them to "forget" user identity even when explicitly stated in the prompt. The system prompt tells the AI the user name, but weaker instruction-following models may override this with generic responses. A secondary contributing factor is that the system prompt injects the user name but does not include an explicit instruction like "The user speaking to you right now IS `${firstName}` - they are the same person as the client/lead listed above."

2.4 Issue #4: Vision API 400 Error

On June 2 at 18:00, when the user sent "Generate a picture of god Vishnu" along with an image attachment, the AI returned: "I encountered an issue analyzing the image: Vision API returned status 400: messages[1].content must be a string." This error indicates that the model receiving the vision request does not support the multimodal content format (array of objects with `type` and `image_url` fields), and instead expects a plain string content. Looking at the code flow: when images are attached, `visionCompletion()` is called at line 523 with a multimodal content array. The vision model is resolved via `getVisionModel()` at line 447. However, the auto-routing logic at lines 449-472 may switch the active model to a different provider that does not support multimodal input. The fallback chain in `callWithFallback()` at line 543-591 can further switch the model on error, potentially selecting a non-vision model.

The critical gap is that `callWithFallback()` does not check whether the fallback model supports vision. It only checks the `tools` feature flag (line 561), not the vision flag. When the fallback system tries DeepSeek V3.1 or Llama 3.3 70B (neither of which supports vision), the API rejects the multimodal content array with a 400 error. Additionally, the `getFallbackModels()` function at line 722 accepts an optional `requiredFeatures` parameter, but the `callWithFallback` function does

not pass vision: true when handling image attachments. This means the fallback chain will happily switch to a non-vision model, causing the 400 error.

2.5 Issue #5: Spirituality Hallucination

On June 2 at 18:01, when the user asked "what is karma space", the AI launched into a 5-paragraph explanation of "Karma Space" as a "virtual or metaphysical realm where the principles of karma are believed to operate" - complete with discussions of energetic realms, interconnectedness, causality, timelessness, and multidimensionality. This directly violates the system prompt rule #1 at line 770: "Never discuss spirituality, religion, or metaphysical concepts of karma." The prompt explicitly states: "Karma Space is an agentic AI assistant within KarmaBoard" and "Karma in KarmaBoard refers to the concept of tracking work actions and their outcomes." The correct answer should have been: "Karma Space is the AI assistant within KarmaBoard that helps you generate project documentation and manage your projects."

The root cause is the same as Issues #3 and #6: the model responding was a fallback model (likely DeepSeek V3.1 or a SambaNova model) that does not reliably follow system prompt instructions. DeepSeek V3.1, despite its high parameter count, has been observed to override explicit system instructions when the user query triggers a strong association in its training data. The phrase "karma space" has a strong association with spiritual/mystical concepts in the training data, which overrides the system prompt instruction. The anti-hallucination filter at lines 744-767 of route.ts does not catch spirituality-related hallucinations because it only targets fake action claims (pushed code, deployments), not content-topic violations.

2.6 Issue #6: Fake Project Creation with Wrong Data

On June 3 at 08:42, the AI stated: "Created project Share Sathi with MEDIUM priority. Deadline: 2024-03-31." This has three separate problems. First, the AI claimed to create a project that already existed (the Share Sathi project was already created, as evidenced by the chat header showing "Status: ACTIVE"). Second, the deadline is wrong: the existing project has a deadline of 2026-07-01, but the AI used 2024-03-31, a date nearly 2 years in the past at the time of the conversation. Third, the AI called the `create_project` tool (indicated by the tool execution happening), but the tool accepted the wrong deadline without validation, leading to a duplicate project with incorrect data in the database.

The root cause has two components. On the AI model side: the AI hallucinated the deadline instead of using the actual project deadline provided in the system prompt context (line 799-808 of `ai-prompts.ts` injects the project deadline). The fallback model did not cross-reference the system prompt context when calling the tool. On the tool executor side: the `createProject()` function in `ai-tool-executor.ts` performs no input validation beyond basic type checking. It does not check for duplicate project names, validate that deadlines are in the

future, or verify that the project does not already exist. This allows the AI to create duplicate projects with fabricated data.

2.7 Issue #7: Generic, Low-Quality Documents

Across all document generations (/docs, /ux, /plan, /schema), the output is consistently generic and uses vague language like "React or Angular", "Python or Java", "MySQL or MongoDB", and "Express or Django" instead of making specific technology recommendations. The project description is only "Creating a website to fetch nepse daily updates and post the update on facebook page" - a single sentence that provides minimal context for generating detailed technical documents. The AI fills in the gaps with generic boilerplate content rather than asking clarifying questions or making specific recommendations based on the project requirements.

The root cause is primarily insufficient project context. The system prompt injects only 7 fields: name, description, client, status, priority, deadline, and team count. For document generation, this is extremely sparse. The AI has no information about preferred tech stack, deployment target, design preferences, or specific features. Additionally, the document prompts instruct the AI to "START WRITING IMMEDIATELY" (line 913 of ai-prompts.ts), which discourages the AI from asking clarifying questions before generating. When combined with a weak fallback model, the result is generic boilerplate that could apply to any project.

3. Root Cause Deep Dive: Code-Level Analysis

3.1 The Primary Root Cause: DeepSeek V3.1 Fallback Priority

The single most impactful root cause across all 7 issues is the DeepSeek V3.1 model being ranked as the highest-priority fallback in both the auto-routing quality order and the fallback quality order. In `ai-models.ts` line 643, the quality order defines "DeepSeek-V3.1": 1, making it the #1 priority model. Similarly, in the `getFallbackModels()` function at line 735, "DeepSeek-V3.1": 1 is again ranked first. This means that whenever the primary model (GLM-4-Flash for docs, or the global default model for regular chat) fails with a 4xx/5xx error, the system immediately tries DeepSeek V3.1.

DeepSeek V3.1 is a 671-billion parameter Mixture of Experts model served via SambaNova. While its raw benchmark scores are impressive, it exhibits several problematic behaviors in the Karma Space context. First, it has poor instruction-following reliability for complex system prompts - it tends to override system instructions when the user query triggers strong associations in its training data. Second, it generates non-standard Unicode characters that cause mojibake in stored content. Third, it fabricates actions (claiming to push code, create repos) even when no tool calls are made. Fourth, it has no vision capability, yet the fallback chain does not filter it out for vision requests. The model was likely ranked #1 because of its benchmark quality, but benchmark scores do not reliably predict instruction-following reliability in production agentic systems.

3.2 Weak Anti-Hallucination Filter

The anti-hallucination filter at lines 744-767 of `route.ts` is a post-processing step that uses 3 regex patterns to detect fake action claims. The patterns catch: (1) claims about pushing/committing/deploying code, (2) stalling phrases like "stand by" or "please wait", and (3) fake build/deployment success messages. These patterns are then replaced with note strings like "[Note: This action was not actually performed.]". While this filter catches the most egregious hallucinations, it has severe limitations. It only triggers when NO git/file tools were executed (line 758), meaning if ANY tool was called (even an unrelated one like `list_projects`), the filter is completely disabled. It uses hardcoded project names like "Share Sathi" (line 764), which is not scalable and only catches that specific project name. Most importantly, it cannot detect hallucinations about project details, wrong deadlines, spiritual content, or garbled text.

3.3 Vision Fallback Lacks Feature Validation

The `callWithFallback()` function at line 543 of `route.ts` accepts feature flags for tools but NOT for vision. At line 561, it calls `getFallbackModels(model, {tools: !!tools, vision: false})` - notice that `vision` is hardcoded to `false`. This means when a vision request fails and the fallback chain activates, it will try non-vision models like DeepSeek V3.1 or Llama 3.3 70B, which cannot handle multimodal content and will return a 400 error. The `getFallbackModels()` function already supports vision filtering (line 759: `if (requiredFeatures?.vision and !m.supportsVision) return false`), but the caller never passes this flag. This is a one-line fix that would prevent all vision-related 400 errors.

3.4 No Tool Input Validation

The tool executor in `ai-tool-executor.ts` performs RBAC checks and basic type validation, but does not validate the semantic correctness of tool inputs. For `create_project`, it does not check for: duplicate project names (the same project can be created multiple times), past deadlines (a deadline of 2024-03-31 is accepted even when it is in the past), empty descriptions (a one-word description is accepted), or missing required fields. For `github_push_code`, it does not validate that there are actually staged files to push. This means any hallucinated tool call from the AI model will be executed without validation, potentially creating duplicate projects, pushing empty commits, or saving incorrect GitHub configurations.

3.5 Insufficient System Prompt Enforcement for Fallback Models

The system prompt in `ai-prompts.ts` contains 15 rules (line 769-791) that are well-designed for instruction-following models like GLM-4-Flash and GPT-4o. However, fallback models like DeepSeek V3.1 and Llama 3.3 70B have varying levels of instruction-following capability. The current approach relies on the model to self-enforce all 15 rules, with only a weak post-processing filter as a safety net. For the most critical rules (anti-hallucination of actions, project scoping, and content topic restrictions), the system should not rely solely on the model but should also implement server-side validation. For example, instead of relying on the model to not fabricate GitHub pushes, the system should verify that a `github_push_code` tool call actually occurred before allowing the response to mention any push-related language.

4. Permanent Solutions

4.1 Solution 1: Demote DeepSeek V3.1 and Restructure Fallback Priority

Impact: Resolves Issues #2, #3, #5, #6, #7 (5 of 7 issues). This is the highest-impact fix.

Move DeepSeek V3.1 from priority 1 to priority 6 or lower in both the `qualityOrder` object in `findBestModelForPrompt()` (line 643) and the `fallbackQuality` object in `getFallbackModels()` (line 735). Replace it with `glm-4-flash` as the top fallback, since GLM-4-Flash is free, has 128K context, 16K output, supports tools, and has excellent instruction-following. The new priority should be: (1) `glm-4-flash`, (2) `gpt-4o`, (3) `Meta-Llama-3.3-70B-Instruct`, (4) `Llama-4-Maverick`, (5) `gpt-4o-mini`, (6) `DeepSeek-V3.1`, (7) `llama-3.3-70b-versatile`. Additionally, add a reliability blacklist for models known to hallucinate in agentic contexts, so they are only used as a last resort.

4.2 Solution 2: Fix Vision Fallback Feature Flag

Impact: Resolves Issue #4 (Vision API 400 error).

In `route.ts` line 561, change `vision: false` to `vision: hasImages`. This is a one-line fix that ensures the fallback chain only considers vision-capable models (GPT-4o, GPT-4o-mini, Gemini 2.0 Flash, Gemini 1.5 Flash, Gemini 1.5 Pro, GLM-4.6V) when processing image attachments. This prevents the system from switching to DeepSeek V3.1 or Llama models that cannot handle multimodal content.

4.3 Solution 3: Add Server-Side Tool Input Validation

Impact: Resolves Issue #6 (Fake project creation) and prevents future data corruption.

Add validation in `ai-tool-executor.ts` for critical tool inputs. For `create_project`: check if a project with the same name already exists and return an error if so; validate that the deadline is at least 7 days in the future; require a description of at least 10 characters. For `github_push_code`: verify that staged files exist before attempting to push. For `save_github_config`: validate that the PAT format looks correct before saving. These server-side validations act as a safety net regardless of which model is generating the tool calls.

4.4 Solution 4: Enhance Anti-Hallucination Filter

Impact: Reduces impact of Issue #2 (garbled text), #5 (spirituality), #6 (fake actions).

Expand the anti-hallucination filter beyond 3 regex patterns. Add: (1) Unicode sanitization to replace non-standard characters with their closest ASCII equivalent before storing content; (2) Content-topic validation that checks for spirituality keywords ("metaphysical", "spiritual", "karma space is", "energetic realm") and replaces them with a redirect to the correct KarmaBoard definition; (3) Project name validation that ensures any mentioned project name matches the current project context; (4) Action-claim verification that checks if the response mentions any tool-executable action and verifies it against actual tool execution records. Move from a simple regex filter to a multi-stage validation pipeline.

4.5 Solution 5: Improve Project Context for Document Generation

Impact: Resolves Issue #7 (generic/low-quality documents).

Expand the project context injected into the system prompt for document generation. Currently, only 7 fields are injected. Add: team member names and roles, any previously generated documents (as summaries), the project description with more detail, client requirements if available, and any chat history where the user has expressed preferences. Additionally, modify the document generation prompt to instruct the AI to ask clarifying questions BEFORE generating if the project context is sparse (less than 50 words of description). This gives the AI permission to ask questions rather than generating generic content.

4.6 Solution 6: Strengthen User Identity in System Prompt

Impact: Resolves Issue #3 (AI ignores user identity).

Add an explicit identity statement to the system prompt that is harder for fallback models to ignore. Instead of just saying "Always address the user by their first name: Kiran", add: "CRITICAL IDENTITY: The person you are talking to RIGHT NOW is `${firstName}` `${lastName}` (`${roleLabel}`). They are the SAME person as the client/lead listed in the project context above. If they ask who they are, tell them their full name." Place this identity statement at the very beginning of the system prompt, before any other instructions, since models tend to follow instructions placed earlier in the prompt more reliably.

4.7 Solution 7: Add Model Reliability Scoring

Impact: Prevents future issues from unreliable models across all projects.

Implement a per-model reliability score that tracks success rates, hallucination rates, and user feedback. After each AI response, log the model used, whether tool calls were made (and validated), and whether the user accepted or rejected the response. Over time, models with high hallucination rates are automatically

deprioritized in the fallback chain. This creates a feedback loop that adapts the model selection to actual production behavior rather than static benchmark scores. Store these scores in the Settings table and update them after each interaction. Display model reliability in the admin settings UI.

5. Implementation Priority Matrix

The following matrix provides a recommended implementation order based on impact, effort, and dependencies. Solutions that resolve multiple issues and require minimal code changes should be implemented first.

Pri orit y	Solution	Effor t	Issues Fixed	Files Changed
P0	Demote DeepSeek V3.1	15 min	#2, #3, #5, #6, #7	ai-models.ts
P0	Fix vision fallback flag	2 min	#4	route.ts (1 line)
P1	Tool input validation	2 hours	#6	ai-tool-executor.ts
P1	Enhanced anti-hallucination	3 hours	#2, #5, #6	route.ts
P2	Improved project context	4 hours	#7	route.ts, ai-prompts.ts
P2	Strengthened user identity	30 min	#3	ai-prompts.ts
P3	Model reliability scoring	8 hours	All (long-term)	route.ts, new files

Table 2: Implementation Priority Matrix

The P0 fixes (Solution 1 and Solution 2) can be implemented in under 20 minutes and will resolve 6 of the 7 identified issues. The P1 fixes add critical safety nets that prevent future data corruption and reduce hallucination impact. The P2 and P3 fixes are quality-of-life improvements that enhance the user experience but are not strictly necessary to prevent the issues observed in the Share Sathi chat history. The recommended approach is to implement P0 immediately, then P1 within the same week, and schedule P2/P3 for the following sprint.

6. Conclusion

The Share Sathi chat history reveals that the Karma Space AI system has a fundamental architectural vulnerability: its model fallback chain prioritizes a model (DeepSeek V3.1) that consistently fails to follow system prompt instructions, fabricates actions, generates garbled text, and provides generic output. The anti-hallucination filter, while well-intentioned, is too narrow and too

late (post-processing vs. prevention) to catch the majority of hallucination patterns. The vision fallback lacks feature validation, causing 400 errors when image attachments trigger a model switch. Tool input validation is absent, allowing hallucinated data to be written to the database.

The good news is that the codebase is well-structured and the fixes are straightforward. The two P0 solutions (demoting DeepSeek V3.1 and fixing the vision fallback flag) require changing approximately 4 lines of code across 2 files and will immediately resolve 6 of the 7 identified issues. The remaining solutions add defense-in-depth layers that will prevent similar issues from recurring with any model. The system already has the right architecture (agentic tool-calling, provider fallback, anti-hallucination filter) - it just needs better tuning of the model priority and stronger validation layers.